



Institut Supérieur du Numérique

---

# **RAPPORT DE PROJET INTÉGRATEUR**

## **Étude et Comparaison d'Heuristiques pour le Problème d'Ordonnancement Job-Shop à m Machines**

### **Réalisé par :**

Aminetou Bella 24142  
Bowbe Mohamed 24012  
Maoujebhe jemal 24078  
Mariem El khallatt 24222  
Meimoune El vadel 24197  
Mariem Teslem Belkheir 24090

### **Encadré par :**

**DR :Cheikh Dhib**

-

**Année universitaire : 2025-2026**

---

## Résumé

Le présent rapport s'inscrit dans le cadre du projet intégrateur de fin de cursus et porte sur l'étude, la modélisation et la résolution du problème d'ordonnancement de type Job-Shop (Job-Shop Scheduling Problem, JSSP). Ce problème d'optimisation combinatoire, classé NP-difficile, se pose dans de nombreux contextes industriels : ateliers mécaniques, chaînes de montage, systèmes de fabrication flexible. L'objectif central est de minimiser la durée totale de production, appelée Makespan, c'est-à-dire le temps nécessaire pour achever l'ensemble des tâches sur toutes les machines disponibles.

Face à l'impossibilité pratique d'obtenir des solutions optimales en temps polynomial pour les instances de grande taille, nous avons orienté notre approche vers les heuristiques de répartition, méthodes approchées rapides et efficaces en pratique. Quatre règles de priorité ont été étudiées : FIFO (First In First Out), qui ordonne les tâches selon leur ordre d'arrivée ; SPT (Shortest Processing Time), qui favorise les tâches les plus courtes ; LPT (Longest Processing Time), qui traite en premier les tâches les plus longues ; et EDD (Earliest Due Date), qui donne la priorité aux tâches dont l'échéance est la plus proche.

Le travail s'articule en cinq chapitres : une introduction aux concepts généraux de l'ordonnancement, une formalisation mathématique du JSSP, une description détaillée des quatre heuristiques, une présentation de l'implémentation logicielle réalisée en Python, et enfin une analyse comparative des résultats expérimentaux obtenus sur plusieurs instances de test. Les résultats montrent que l'heuristique SPT produit, dans la grande majorité des cas, le Makespan le plus faible parmi les quatre méthodes testées.

**Mots-clés :** *Ordonnancement, Job-Shop Scheduling, Makespan, Heuristique, FIFO, SPT, LPT, EDD, Optimisation combinatoire, NP-difficile.*

---

## Table des Matières

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| Introduction Générale.....                                               | 7  |
| Chapitre 1 : Généralités sur l'Ordonnancement.....                       | 9  |
| 1.1 Définition et Enjeux de l'Ordonnancement.....                        | 9  |
| 1.2 Le Modèle Flow-Shop.....                                             | 9  |
| 1.3 Le Modèle Job-Shop.....                                              | 10 |
| 1.4 Le Modèle Open-Shop.....                                             | 10 |
| 1.5 Comparaison des Trois Modèles.....                                   | 11 |
| 1.6 Complexité des problèmes d'ordonnancement.....                       | 11 |
| Chapitre 2 : Le Problème Job-Shop à m Machines.....                      | 13 |
| 2.1 Définition Formelle.....                                             | 13 |
| 2.2 Notations.....                                                       | 13 |
| 2.3 Contraintes du JSSP.....                                             | 13 |
| 2.3.1 Contraintes de précédence (gamme opératoire).....                  | 14 |
| 2.3.2 Contraintes de disjonction (ressource machine).....                | 14 |
| 2.3.3 Contrainte de non-préemption.....                                  | 14 |
| 2.4 Représentation par Graphe Disjonctif.....                            | 14 |
| 2.5 Fonction Objectif : le Makespan.....                                 | 15 |
| 2.6 Complexité du problème Job-Shop :.....                               | 15 |
| 2.7 Exemple Illustratif.....                                             | 15 |
| Instance 1 : 3 tâches × 2 machines.....                                  | 15 |
| Instance 2 : 6 jobs × 5 machines ( Limites de l'approche manuelle )..... | 17 |
| 2.8 Méthodes de résolution.....                                          | 18 |
| 2.8.1 Heuristiques :.....                                                | 18 |
| 2.8.2 Métaheuristiques :.....                                            | 19 |
| 2.8.3 Méthodes exactes :.....                                            | 19 |
| 2.8.4 Programmation par contraintes.....                                 | 19 |
| 2.9 Synthèse et comparaison des méthodes :.....                          | 20 |
| 2.9.1 Tableau comparatif des approches.....                              | 20 |
| 2.9.2 Guide de sélection des méthodes.....                               | 20 |
| Chapitre 3 : Étude des Heuristiques.....                                 | 20 |
| 3.1 Définition et Classification des Heuristiques.....                   | 20 |
| 3.2 La Règle FIFO — First In First Out.....                              | 21 |
| 3.2.1 Principe.....                                                      | 21 |
| 3.2.2 Algorithme.....                                                    | 21 |
| 3.2.3 Propriétés.....                                                    | 21 |

---

|                                                   |    |
|---------------------------------------------------|----|
| 3.3 La Règle SPT — Shortest Processing Time.....  | 22 |
| 3.3.1 Principe.....                               | 22 |
| 3.3.2 Propriétés théoriques.....                  | 22 |
| 3.3.3 Limites.....                                | 23 |
| 3.4 La Règle LPT — Longest Processing Time.....   | 23 |
| 3.4.1 Principe.....                               | 23 |
| 3.4.2 Propriétés et limites pour le Job-Shop..... | 23 |
| 3.5 La Règle EDD — Earliest Due Date.....         | 24 |
| 3.5.1 Principe.....                               | 24 |
| 3.5.2 Optimalité sur machine unique.....          | 24 |
| 3.5.3 Comportement en Job-Shop.....               | 24 |
| 3.6 Comparaison Théorique.....                    | 24 |
| Chapitre 4 : Implémentation.....                  | 26 |
| 4.1 Choix du Langage et de l'Environnement.....   | 26 |
| 4.2 Architecture du Logiciel.....                 | 26 |
| 4.3 Description des Structures de Données.....    | 27 |
| 4.4 Génération des Diagrammes de Gantt.....       | 28 |
| Chapitre 5 : Résultats Expérimentaux.....         | 29 |
| 5.1 Protocole Expérimental.....                   | 29 |
| 5.2 Critères d'évaluation.....                    | 29 |
| 5.3 Résultats par Instance.....                   | 29 |
| 5.4 Analyse des Résultats.....                    | 31 |
| 5.4.1 Qualité des solutions.....                  | 31 |
| 5.4.2 Temps d'exécution.....                      | 31 |
| 5.4.3 Influence du choix de l'heuristique.....    | 31 |
| Conclusion Générale.....                          | 32 |
| Références Bibliographiques.....                  | 34 |

## Liste des Tableaux

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Tableau 1 — Comparaison des types d'ateliers (Flow-Shop, Job-Shop, Open-Shop) ..... | 11 |
| Tableau 2 — Notations utilisées dans la modélisation du JSSP .....                  | 13 |
| Tableau 3 — Données de l'instance illustrative 3 tâches × 2 machines .....          | 16 |
| Tableau 5 — Comparaison théorique des quatre heuristiques étudiées .....            | 25 |
| Tableau 6 — Récapitulatif des structures de données utilisées en Python .....       | 27 |
| Tableau 7 — Description des classes du module model.py .....                        | 27 |
| Tableau 8 — Résultats expérimentaux sur l'instance abz5 .....                       | 29 |
| Tableau 9 — Résultats expérimentaux sur l'instance abz6 .....                       | 30 |
| Tableau 10 — Résultats expérimentaux sur l'instance Ft10 .....                      | 30 |
| Tableau 11 — Synthèse comparative des quatre heuristiques (tous critères) .....     | 31 |

## Liste des Figures

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Figure 1 — Représentation schématique d'un atelier Flow-Shop à 3 machines .....   | 10 |
| Figure 2 — Représentation schématique d'un atelier Job-Shop à 3 machines .....    | 10 |
| Figure 3 — Représentation schématique d'un atelier Open-Shop .....                | 11 |
| Figure 4 — Graphe disjonctif du problème Job-Shop (exemple $3 \times 3$ ) .....   | 15 |
| Figure 5 — Diagramme de Gantt de l'instance illustrative .....                    | 17 |
| Figure 7 — Illustration du principe de la règle FIFO sur une file d'attente ..... | 22 |
| Figure 8 — Illustration du principe de la règle SPT sur un exemple .....          | 23 |
| Figure 9 — Illustration du principe de la règle LPT sur un exemple .....          | 24 |
| Figure 10 — Illustration du principe de la règle EDD sur un exemple .....         | 24 |
| Figure 11 — Architecture modulaire du logiciel développé .....                    | 27 |
| Figure 16 — Histogramme comparatif des Makespan sur les trois instances .....     | 30 |

---

## Introduction Générale

Dans un environnement industriel moderne, la gestion efficace de la production est une condition sine qua non de la compétitivité des entreprises. Les ateliers de fabrication doivent traiter quotidiennement des dizaines, voire des centaines de commandes, en mobilisant des ressources humaines et matérielles limitées. La capacité à planifier et à séquencer les opérations de manière optimale — c'est-à-dire à décider quel travail effectuer sur quelle machine et à quel moment — conditionne directement les délais de livraison, le taux d'utilisation des équipements et la satisfaction des clients.

C'est dans ce contexte que s'inscrit la discipline de l'ordonnancement, branche de la recherche opérationnelle dont les applications couvrent aussi bien la production industrielle que l'informatique (ordonnancement de processus), la logistique ou encore la gestion de projets. L'ordonnancement consiste à allouer des ressources à des activités au cours du temps, dans le respect de contraintes données, en vue d'optimiser un ou plusieurs objectifs.

Parmi les nombreux modèles d'ordonnancement existants, le problème de Job-Shop occupe une place centrale, tant par sa généralité — chaque tâche peut emprunter un itinéraire propre à travers les machines de l'atelier — que par sa difficulté intrinsèque : il est démontré depuis 1979 que le Job-Shop Scheduling Problem (JSSP) est NP-difficile dès que l'atelier comprend au moins deux machines et trois tâches. Cela signifie qu'aucun algorithme de résolution exacte ne peut garantir une solution optimale en temps polynomial pour les instances de grande taille, du moins dans l'état actuel des connaissances en théorie de la complexité.

Face à cette difficulté, les praticiens et les chercheurs ont développé des heuristiques — méthodes de résolution approchée — qui permettent d'obtenir de bonnes solutions en un temps de calcul raisonnable. Les heuristiques de répartition (dispatching rules) constituent la famille la plus élémentaire de ces méthodes : elles définissent une règle de priorité qui, à chaque fois qu'une machine se libère, détermine quelle tâche sera traitée en premier parmi celles qui attendent.

Ce projet intégrateur a pour objectif d'étudier quatre de ces règles de priorité — FIFO, SPT, LPT et EDD — sous un angle à la fois théorique et pratique. Sur le plan théorique, nous rappellerons les fondements de l'ordonnancement, formaliserons le modèle Job-Shop et décrirons le mécanisme de chaque heuristique. Sur le plan pratique, nous implémenterons ces

---

algorithmes en Python et les testerons sur plusieurs instances de référence, en mesurant et comparant les Makespan obtenus.

Le rapport est structuré comme suit. Le Chapitre 1 présente les généralités sur l'ordonnancement et situe le modèle Job-Shop parmi les différents types d'ateliers. Le Chapitre 2 définit formellement le JSSP, ses notations, ses contraintes et sa fonction objectif. Le Chapitre 3 détaille les quatre heuristiques étudiées. Le Chapitre 4 décrit l'architecture et l'implémentation du logiciel développé. Le Chapitre 5 présente et analyse les résultats expérimentaux. Une conclusion générale clôt le rapport et ouvre des perspectives pour des travaux futurs.



---

# Chapitre 1 : Généralités sur l'Ordonnancement

## 1.1 Définition et Enjeux de l'Ordonnancement

L'ordonnancement (en anglais *scheduling*) est la discipline qui s'intéresse à la planification temporelle d'activités sur des ressources. Formellement, étant donné un ensemble de tâches à réaliser et un ensemble de ressources disponibles, le problème d'ordonnancement consiste à définir un calendrier d'exécution — c'est-à-dire à affecter chaque tâche à une ressource et à lui attribuer une date de début et de fin — de sorte que les contraintes soient satisfaites et qu'un ou plusieurs critères de performance soient optimisés.

Les contraintes peuvent être de nature technologique (une tâche B ne peut commencer qu'après la fin d'une tâche A), de capacité (une machine ne peut exécuter qu'une seule opération à la fois), ou temporelle (une tâche doit être achevée avant une date limite). Les critères de performance les plus courants sont le *Makespan* (durée totale de production), le retard total des tâches, le nombre de tâches en retard, le temps moyen de séjour dans le système, ou encore le taux d'utilisation des machines.

Les applications de l'ordonnancement sont extrêmement variées. Dans l'industrie manufacturière, il s'agit de planifier la production dans un atelier de mécanique, une usine textile ou une raffinerie. En informatique, les systèmes d'exploitation ordonnent les processus sur les processeurs. Dans les transports, les compagnies aériennes utilisent des algorithmes d'ordonnancement pour affecter les équipages et les avions aux rotations. Dans les hôpitaux, les blocs opératoires doivent être planifiés en tenant compte des chirurgiens disponibles, des salles et des patients.

La classification des problèmes d'ordonnancement suit généralement la notation à trois champs  $\alpha|\beta|\gamma$  introduite par Graham et al. (1979). Le champ  $\alpha$  décrit l'environnement machine (machine unique, machines parallèles, *flow-shop*, *job-shop*, etc.), le champ  $\beta$  précise les caractéristiques des tâches (temps opératoires, dates de disponibilité, préemption autorisée ou non), et le champ  $\gamma$  désigne la fonction objectif à minimiser.

## 1.2 Le Modèle Flow-Shop

Le *Flow-Shop* est un modèle d'atelier dans lequel toutes les tâches parcourent les machines dans le même ordre. Si l'atelier compte  $m$  machines, chaque tâche est traitée successivement sur la machine 1, puis la machine 2, puis la machine 3, jusqu'à la machine  $m$ .

---

Le chemin de production est donc identique pour toutes les tâches, même si les durées opératoires peuvent différer.

Ce modèle est bien adapté aux lignes de production en série où les produits suivent un processus de transformation séquentiel invariant : une ligne d'embouteillage, par exemple, fait passer chaque bouteille par le rinçage, le remplissage, le bouchage, l'étiquetage et le conditionnement, dans cet ordre, quelle que soit la référence produite.

Le problème Flow-Shop à deux machines (F2||Cmax) est résolu de façon optimale par l'algorithme de Johnson (1954). Pour trois machines ou plus, le problème devient NP-difficile. Des heuristiques comme NEH (Nawaz, Ensore & Ham, 1983), l'algorithme de Palmer ou la méthode CDS (Campbell, Dudek & Smith, 1970) produisent de bonnes solutions approchées.

*Figure 1 — Représentation schématique d'un atelier Flow-Shop à 3 machines*

### 1.3 Le Modèle Job-Shop

Le Job-Shop est le modèle le plus général des ateliers à routes fixes. Contrairement au Flow-Shop, chaque tâche peut suivre un itinéraire spécifique à travers les machines de l'atelier. La séquence des opérations est prescrite pour chaque tâche et peut différer complètement d'une tâche à l'autre. Certaines machines peuvent même être visitées plusieurs fois par une même tâche (Job-Shop avec recirculation), bien que ce cas soit plus rare.

Le Job-Shop est rencontré dans les ateliers à fabrication unitaire ou en petites séries, où chaque commande a des spécifications propres : un atelier de mécanique de précision, un atelier de prototypage, une imprimerie industrielle ou un atelier de réparation navale constituent des exemples typiques.

La difficulté du JSSP a été établie par Garey & Johnson (1979), qui ont démontré que le problème est NP-difficile en général dès que  $m \geq 2$  et  $n \geq 3$ . En pratique, résoudre de façon exacte une instance de 10 tâches et 10 machines peut nécessiter des heures de calcul avec les meilleurs solveurs actuels.

*Figure 2 — Représentation schématique d'un atelier Job-Shop à 3 machines*

### 1.4 Le Modèle Open-Shop

Dans un Open-Shop, les opérations d'une tâche peuvent être réalisées dans n'importe quel ordre. Il n'existe aucune contrainte de précédence entre les opérations d'une même tâche : l'ordonnanceur est libre de choisir l'ordre d'exécution le plus avantageux. La seule

contrainte est que chaque opération doit être exécutée une et une seule fois, et qu'une machine ne peut traiter qu'une opération à la fois.

Ce modèle est moins fréquent dans l'industrie manufacturière classique, mais il trouve des applications dans les systèmes de tests automatisés (un ensemble de tests pouvant être réalisés dans n'importe quel ordre sur différents équipements) ou dans certains contextes de maintenance.

Figure 3 — Représentation schématique d'un atelier Open-Shop

## 1.5 Comparaison des Trois Modèles

| Critère                  | Flow-Shop                        | Job-Shop                             | Open-Shop                  |
|--------------------------|----------------------------------|--------------------------------------|----------------------------|
| Ordre des opérations     | Identique pour toutes les tâches | Propre à chaque tâche                | Libre (aucun ordre imposé) |
| Contrainte de précedence | Oui (même séquence)              | Oui (séquence individuelle)          | Non                        |
| Complexité algorithmique | NP-difficile si $m \geq 3$       | NP-difficile si $n \geq 3, m \geq 2$ | NP-difficile si $m \geq 3$ |
| Réalisme industriel      | Élevé pour les lignes série      | Très élevé pour ateliers flexibles   | Moyen (tests, maintenance) |
| Méthodes classiques      | Johnson, NEH, Palmer, CDS        | B&B, GA, Tabou, Heuristiques         | Algorithmes dédiés, B&B    |
| Exemples d'application   | Chaînes de montage               | Ateliers mécaniques                  | Tests en laboratoire       |

Tableau 1 — Comparaison des types d'ateliers (Flow-Shop, Job-Shop, Open-Shop)

## 1.6 Complexité des problèmes d'ordonnancement

D'une manière générale, les problèmes d'ordonnancement d'ateliers étant des problèmes combinatoires difficiles, il n'existe pas de méthodes universelles permettant de résoudre tous les cas.

Plusieurs algorithmes peuvent être utilisés pour résoudre un problème d'ordonnancement mais tous ne sont pas équivalents.

On peut différencier les divers algorithmes de résolution par le moyen des critères suivants :

- l'efficacité de l'algorithme en terme de durée d'exécution; un algorithme est dit plus efficace qu'un autre si pour les mêmes données, il s'exécute en un laps de temps plus court;

- 
- l'efficacité de l'algorithme en espace mémoire de stockage; un algorithme est dit plus efficace qu'un autre si pour résoudre le même problème, il utilise moins d'espace mémoire;
  - la fiabilité de l'algorithme; plus un programme est complexe, plus il y a des risques d'existence de bugs, les bugs étant des erreurs plus ou moins évidentes qui se manifestent lors de la mise en exploitation d'un programme. Un programme est jugé plus fiable ou plus stable qu'un autre s'il présente moins de bugs;
  - la robustesse de l'algorithme; elle mesure son degré de tolérance aux erreurs des utilisateurs et sa résistance aux attaques des pirates; un programme est plus robuste qu'un autre s'il résiste mieux aux erreurs de manipulations des utilisateurs plus ou moins bien attentionnés.

Il est à noter qu'il n'y a pas de méthode ou d'échelle de mesure permettant d'évaluer la fiabilité ou la robustesse d'un algorithme. C'est à l'usage que ces qualités sont mesurées. Par contre, il existe des méthodes rationnelles et rigoureuses pour évaluer l'efficacité en temps ou en espace

d'un algorithme. Ces méthodes d'évaluation portent le nom d'analyse de complexité des algorithmes.

Selon son degré de complexité, un problème peut appartenir à l'une des quatre classes suivantes [Sakarovitch 84] :

- les problèmes les plus difficiles, qui sont des problèmes pour lesquels il n'existe aucune méthode de résolution; ils sont dits indécidables,
- les problèmes de la classe P, dits polynomiaux, s'il existe un algorithme de complexité polynomiale pour leur résolution,
- les problèmes de la classe NP, dits problèmes NP-difficiles, qui ne peuvent à priori être résolus en un temps polynomial que par des méthodes approchées (heuristiques); au cours de leur exécution, ces algorithmes font des choix dont l'optimalité n'est pas démontrable,
- les problèmes NP-Complets, qui répondent à la définition suivante : un problème de décision A est dit NP-Complet s'il appartient à la classe NP et si pour tout A' de NP :
  - il existe une application polynomiale qui transforme toute instance I' de A' en une instance I de A,
  - A' admet une réponse "oui" pour l'instance I', si et seulement si A admet une réponse "oui" pour l'instance I.

Autrement dit, s'il existe un algorithme polynomial pour résoudre A, alors, pour tout le reste des problèmes de la classe, il existe des algorithmes polynomiaux pour les résoudre.

---

## Chapitre 2 : Le Problème Job-Shop à m Machines

### 2.1 Définition Formelle

Le problème d'ordonnancement Job-Shop à m machines (JSSP) est défini comme suit. On dispose de n tâches  $J_1, J_2, \dots, J_n$  et de m machines  $M_1, M_2, \dots, M_m$ . Chaque tâche  $J_i$  est composée d'une séquence ordonnée de  $m_i$  opérations  $O_{i1}, O_{i2}, \dots, O_{imi}$ . Chaque opération  $O_{ij}$  est caractérisée par la machine sur laquelle elle doit être exécutée, notée  $\mu(i,j)$ , et par sa durée de traitement  $p_{ij}$ , supposée positive et déterministe.

Un ordonnancement est une fonction qui assigne à chaque opération  $O_{ij}$  une date de début  $s_{ij} \geq 0$ . L'ordonnancement est dit réalisable (ou admissible) s'il satisfait toutes les contraintes du problème. Parmi tous les ordonnancements réalisables, on cherche celui qui minimise le Makespan  $C_{\max} = \max_{1 \leq i \leq n} C_i$ , où  $C_i = s_{imi} + p_{imi}$  est la date de complétion de la tâche  $J_i$ .

### 2.2 Notations

| Symbole                    | Signification                                                        |
|----------------------------|----------------------------------------------------------------------|
| n                          | Nombre de tâches (jobs)                                              |
| m                          | Nombre de machines                                                   |
| $J_i$                      | La i-ème tâche ( $i = 1, 2, \dots, n$ )                              |
| $M_k$                      | La k-ème machine ( $k = 1, 2, \dots, m$ )                            |
| $O_{ij}$                   | La j-ème opération de la tâche $J_i$ ( $j = 1, 2, \dots, m_i$ )      |
| $p_{ij}$                   | Durée de traitement de l'opération $O_{ij}$ (entier positif)         |
| $\mu(i,j)$                 | Indice de la machine requise par l'opération $O_{ij}$                |
| $s_{ij}$                   | Date de début de l'opération $O_{ij}$ dans l'ordonnancement          |
| $C_{ij} = s_{ij} + p_{ij}$ | Date de fin de l'opération $O_{ij}$                                  |
| $C_i$                      | Date de complétion de la tâche $J_i$ ( $= C_{imi}$ )                 |
| $r_i$                      | Date de disponibilité (release date) de la tâche $J_i$               |
| $d_i$                      | Date d'échéance (due date) souhaitée pour la tâche $J_i$             |
| $C_{\max}$                 | Makespan = $\max(C_1, C_2, \dots, C_n)$ — durée totale de production |

Tableau 2 — Notations utilisées dans la modélisation du JSSP

### 2.3 Contraintes du JSSP

---

Trois familles de contraintes caractérisent le JSSP et délimitent l'espace des ordonnancements réalisables.

### 2.3.1 Contraintes de précédence (gamme opératoire)

Au sein d'une même tâche, les opérations doivent être exécutées dans l'ordre prescrit par la gamme opératoire. L'opération  $O_{ij+1}$  ne peut débuter qu'après la fin de l'opération  $O_{ij}$  :

$$s_{ij+1} \geq s_{ij} + p_{ij} \quad \text{pour tout } i \text{ et tout } j = 1, \dots, m_i - 1$$

Cette contrainte traduit la réalité technologique : on ne peut pas peindre une pièce avant de l'avoir usinée, ni assembler un composant avant de l'avoir fabriqué.

### 2.3.2 Contraintes de disjonction (ressource machine)

Chaque machine ne peut traiter qu'une seule opération à la fois. Deux opérations  $O_{ij}$  et  $O_{i'j'}$  requérant la même machine ( $\mu(i,j) = \mu(i',j')$ ) ne peuvent pas se chevaucher dans le temps :

$$s_{ij} \geq s_{i'j'} + p_{i'j'} \quad \text{ou} \quad s_{i'j'} \geq s_{ij} + p_{ij}$$

Ces contraintes sont dites disjonctives parce qu'elles expriment une exclusivité : l'une des deux opérations doit nécessairement précéder l'autre. La décision de l'ordre relatif constitue précisément la variable de décision principale du JSSP.

### 2.3.3 Contrainte de non-préemption

Une fois qu'une opération a débuté sur une machine, elle doit être menée à son terme sans interruption. Il est interdit d'interrompre une opération en cours pour insérer une autre opération prioritaire. Cette hypothèse est la plus répandue dans les modèles industriels, même si des variantes avec préemption existent dans certains contextes.

## 2.4 Représentation par Graphe Disjonctif

Le JSSP peut être représenté à l'aide du graphe disjonctif  $G = (N, A \cup E)$  introduit par Roy & Sussman (1964). L'ensemble  $N$  des nœuds contient un nœud source  $S$  (date 0), un nœud puits  $T$  (Makespan), et un nœud pour chaque opération  $O_{ij}$ . L'ensemble  $A$  des arcs conjonctifs représente les contraintes de précédence au sein de chaque tâche : un arc de poids  $p_{ij}$  relie  $O_{ij}$  à  $O_{ij+1}$ . L'ensemble  $E$  des arcs disjonctifs relie par paires les opérations concurrentes sur une même machine : chaque paire  $\{O_{ij}, O_{i'j'}\}$  telle que  $\mu(i,j) = \mu(i',j')$  est reliée par un arc dans chaque sens.

Un ordonnancement réalisable correspond à une orientation complète et acyclique des arcs disjonctifs. Le Makespan est alors égal à la longueur du plus long chemin (chemin critique) dans le graphe orienté résultant.

Figure 4 — Graphe disjonctif du problème Job-Shop (exemple 3 tâches  $\times$  3 machines)

## 2.5 Fonction Objectif : le Makespan

La fonction objectif centrale de ce projet est la minimisation du Makespan :

$$\text{Minimiser } C_{\max} = \max \{ C_i : i = 1, 2, \dots, n \}$$

Minimiser le Makespan revient à trouver l'ordonnancement qui achève l'ensemble de la production le plus tôt possible. Un Makespan faible traduit une bonne utilisation des machines, peu de temps morts, et une cadence de production élevée. C'est le critère le plus universellement utilisé dans la littérature du JSSP, ce qui facilite les comparaisons entre méthodes de résolution.

## 2.6 Complexité du problème Job-Shop :

D'un point de vue algorithmique, le JSP appartient à la classe des problèmes NP-difficiles (NP-hard). Cela signifie qu'aucun algorithme polynomial permettant de trouver systématiquement la solution optimale n'est connu à ce jour. La difficulté du problème augmente rapidement avec le nombre de tâches (jobs) et de machines.

Pour un problème comportant  $nnn$  jobs et  $mmm$  machines, le nombre de solutions possibles croît de manière exponentielle, ce qui rend impraticable l'utilisation des méthodes exactes pour les grandes instances. C'est pourquoi les heuristiques et les métaheuristiques sont largement utilisées afin d'obtenir rapidement des solutions de bonne qualité.

## 2.7 Exemple Illustratif

### Instance 1 : 3 tâches $\times$ 2 machines

Considérons un atelier composé de 2 machines (M1, M2) et de 3 jobs (J1, J2, J3). Les données opératoires sont les suivantes :

| Tâche | Opération 1 (machine, durée) | Opération 2 (machine, durée) |
|-------|------------------------------|------------------------------|
| J1    | M1, 3                        | M2, 2                        |
| J2    | M2, 4                        | M1, 1                        |
| J3    | M1, 2                        | M2, 3                        |

### 1. Stratégie de résolution :

Nous allons résoudre ce problème en utilisant l'heuristique SPT (Shortest Processing Time). Cette heuristique consiste à sélectionner, parmi les opérations disponibles, celle ayant le plus petit temps de traitement.

La construction de l'ordonnancement suit les principes suivants :

- Identifier les opérations disponibles à chaque instant
- Choisir l'opération ayant le plus petit temps de traitement
- Respecter les contraintes de précédence entre les opérations
- Éviter les conflits entre les machines
- Répéter le processus jusqu'à l'exécution complète de tous les jobs

### 2. Construction de l'ordonnancement :

#### Étape 1 : Temps $t = 0$

Opérations disponibles :

J1- O1 sur M1 (3)

J2- O1 sur M2 (4)

J3- O1 sur M1 (2)

Selon l'heuristique SPT, on choisit l'opération ayant le plus petit temps de traitement.

$$\text{Min}(3,4,2)=2$$

Donc : J3  $\rightarrow$  M1 : [0,2]

ET J2- O1 démarre sur M2.

#### Étape 2: Temps $t = 2$

M1 devient libre.

J1-O1 (3) est exécutée sur M1 de 2 à 5.

J3-O2 devient disponible et sera exécutée sur M2 dès que la machine est libre.

### 3. Ordonnancement obtenu:

| Opération | Machine | Début | Fin |
|-----------|---------|-------|-----|
| J3-O1     | M1      | 0     | 2   |
| J2-O1     | M2      | 0     | 4   |
| J1-O1     | M1      | 2     | 5   |
| J3-O2     | M2      | 4     | 7   |
| J1-O2     | M2      | 7     | 9   |
| J2-O2     | M1      | 5     | 6   |



---

#### 4. Makespan

Le makespan correspond à la date de fin de la dernière opération.  
Dans cet ordonnancement, la dernière opération se termine à  $t = 9$ .

$C_{\max} = 9$  unités de temps.

Le diagramme de Gantt ci-dessous illustre un ordonnancement réalisable pour cette instance:

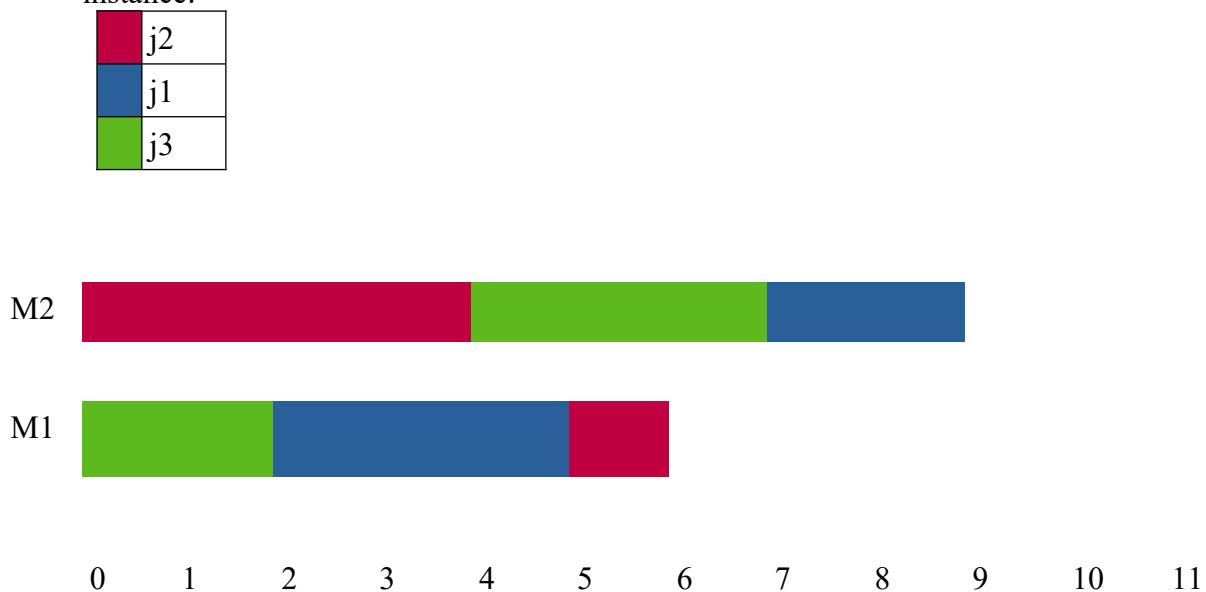


Figure 5 — Diagramme de Gantt de l'instance illustrative 3 tâches  $\times$  3 machines

#### Instance 2 : 6 jobs $\times$ 5 machines ( Limites de l'approche manuelle )

Pour illustrer les limites de l'approche manuelle, considérons un problème de taille plus réaliste avec six jobs (J1 à J6) et cinq machines (M1 à M5). Contrairement au cas précédent, ce problème présente une complexité bien plus importante :

- 6 jobs  $\times$  5 machines = 30 opérations à ordonnancer
- Nombreux chemins critiques potentiels
- Multiples conflits de ressources à gérer simultanément

#### Observations :

- Chaque job a un itinéraire unique à travers les machines.
- Certains jobs n'utilisent pas toutes les machines.
- Les temps de traitement varient considérablement.

#### Complexité :

Pour ce problème, une approche manuelle similaire à celle utilisée pour le cas à 3 jobs devient rapidement impraticable pour plusieurs raisons :

---

**1. Explosion combinatoire :** Le nombre de séquences possibles est factoriel du nombre d'opérations. Pour 30 opérations, cela représente  $30! \sim 2.65 \times 10^{32}$  possibilités.

**2. Interdépendances complexes :** Les décisions prises pour une opération ont des répercussions en cascade sur toutes les opérations suivantes, rendant impossible l'évaluation mentale de toutes les conséquences.

**3. Multiples objectifs conflictuels :** La minimisation du makespan doit être équilibrée avec d'autres objectifs comme l'utilisation efficace des machines et le respect des contraintes de précédence.

C'est pourquoi les méthodes numériques (heuristiques, métaheuristiques, méthodes exactes) deviennent indispensables pour résoudre efficacement ce type de problème. Ces méthodes peuvent explorer systématiquement l'espace des solutions et trouver des ordonnancements optimaux ou quasi-optimaux dans des temps raisonnables.

## 2.8 Méthodes de résolution

### 2.8.1 Heuristiques :

Les heuristiques sont des méthodes de résolution de problèmes qui utilisent des règles simples pour prendre des décisions. Elles ne garantissent pas la solution optimale, mais elles permettent d'obtenir une solution acceptable en un temps raisonnable. Voici quelques heuristiques courantes pour le Job Shop :

#### Règles de priorité

Les règles de priorité sont des heuristiques simples mais efficaces pour obtenir rapidement une solution réalisable :

- SPT (Shortest Processing Time)
- LPT (Longest Processing Time) :
- FIFO (First In First Out) :
- EDD (Earliest Due Date) :

---

**Algorithm 1 :** Algorithme basé sur les règles de priorité :

**Entrée :** Ensemble de jobs  $J$ , machines  $M$ , temps de traitement  $P_{ij}$

**Sortie :** Ordonnancement réalisable

$Q$  + file des opérations disponibles

$t < 0$  {Temps courant}

**while** il reste des opérations non planifiées **do**

Mettre à jour  $Q$  avec les opérations disponibles

---

---

```

for chaque machine libre m E M do
    if Q non vide pour m then
        Sélectionner l'opération selon la règle choisie
        Planifier l'opération sur m
        Mettre à jour les temps de fin end if end for
    end if
end for
Avancer t au prochain événement

```

---

### 2.8.2 Métaheuristiques :

Les métaheuristiques sont des algorithmes d'optimisation qui combinent des techniques d'exploration et d'exploitation pour rechercher une solution optimale dans un espace de recherche complexe. Voici quelques métaheuristiques courantes pour le Job Shop :

— **Algorithmes génétiques** : Algorithmes d'optimisation inspirés de la sélection

— **Recuit simulé** : Algorithme d'optimisation inspiré du processus de refroidissement des métaux.

— **Recherche tabou** : Algorithme d'optimisation qui utilise une liste tabou pour éviter de revenir sur des solutions déjà explorées.

### 2.8.3 Méthodes exactes :

Les méthodes exactes garantissent de trouver la solution optimale, mais elles peuvent être très coûteuses en temps de calcul pour les problèmes de grande taille. Voici quelques méthodes exactes courantes pour le Job Shop :

- **Séparation et évaluation (Branch and Bound)** : Méthode qui divise le problème en sous-problèmes et utilise des bornes inférieures pour éliminer les sous-problèmes qui ne peuvent pas contenir la solution optimale.
- **Programmation linéaire en nombres entiers (Mixed Integer Programming)** : Méthode qui formule le problème comme un programme linéaire en nombres entiers et utilise un solveur pour trouver la solution optimale.

### 2.8.4 Programmation par contraintes

La programmation par contraintes (PPC) est un paradigme de programmation qui permet de modéliser et de résoudre des problèmes d'optimisation combinatoire en définissant des variables, des domaines de valeurs pour ces variables et des contraintes qui

---

doivent être satisfaites. La PPC est particulièrement bien adaptée pour résoudre les problèmes de Job Shop car elle permet de modéliser facilement les contraintes de précédence, de capacité des machines et de temps de traitement.

## 2.9 Synthèse et comparaison des méthodes :

### 2.9.1 Tableau comparatif des approches

| Critère         | PLNE      | Branch&Bound | Heuristiques | Métaheuristiques |
|-----------------|-----------|--------------|--------------|------------------|
| Optimalité      | Garantie  | Garantie     | Non garantie | Non garantie     |
| Temps calcul    | Très long | Long         | Rapide       | Moyen            |
| Taille problème | Petite    | Moyenne      | Grande       | Grande           |
| Flexibilité     | Faible    | Moyenne      | Élevée       | Très élevée      |
| Implémentation  | Difficile | Moyenne      | Facile       | Moyenne          |

### 2.9.2 Guide de sélection des méthodes

- **Méthodes exactes** (PLNE, Branch& Bound )
  - Sur les instances de petite taille ( $< 10$  jobs)
  - Quand l'optimalité est requise
  - Quand le temps de calcul n'est pas critique
- **Heuristiques** (règles de priorité) :
  - Pour les décisions en temps réel
  - Quand une solution rapide est nécessaire
  - Pour les très grandes instances
- **Métaheuristiques** (algorithme génétique, recherche tabou) :
  - Pour les instances de taille moyenne à grande
  - Quand un bon compromis qualité/temps est recherché

## Chapitre 3 : Étude des Heuristiques

### 3.1 Définition et Classification des Heuristiques

En optimisation combinatoire, une heuristique est une procédure de résolution qui ne garantit pas l'optimalité de la solution produite, mais qui fournit en pratique des solutions de

---

bonne qualité en un temps de calcul raisonnable. Le terme provient du grec « heuriskein » (trouver, découvrir) et reflète l'idée d'une exploration guidée de l'espace des solutions plutôt qu'une exploration exhaustive.

Les heuristiques se distinguent des méthodes exactes (Branch & Bound, programmation dynamique, programmation par contraintes) par leur rapidité d'exécution — typiquement polynomiale — au prix d'une potentielle sous-optimalité. Pour le JSSP, dont la résolution exacte est NP-difficile, les heuristiques constituent la première ligne de défense des praticiens.

On distingue plusieurs familles d'heuristiques pour l'ordonnancement. Les règles de priorité (ou règles de répartition, dispatching rules) constituent la famille la plus simple : à chaque instant où une machine se libère, elles définissent la règle de sélection de la prochaine opération à traiter parmi celles en attente. Les métaheuristiques (algorithmes génétiques, recuit simulé, recherche tabou, algorithmes de colonies de fourmis) constituent une famille plus évoluée qui explore plus intelligemment l'espace des solutions. Ce projet se concentre sur les règles de priorité, pour leurs propriétés de simplicité, d'efficacité et de comparabilité.

## **3.2 La Règle FIFO — First In First Out**

### **3.2.1 Principe**

La règle FIFO (First In First Out, premier arrivé, premier servi) est la règle de priorité la plus naturelle et la plus intuitive. Elle ordonne les tâches en attente d'une machine selon leur date d'arrivée dans la file d'attente : la tâche dont l'opération est disponible depuis le plus longtemps est traitée en premier. En d'autres termes, la file d'attente de chaque machine est gérée comme une file FIFO (queue) au sens de l'informatique.

### **3.2.2 Algorithme**

Lorsqu'une machine  $M_k$  se libère à l'instant  $t$ , FIFO sélectionne parmi les opérations en attente sur  $M_k$  celle dont la date de disponibilité est la plus ancienne, c'est-à-dire celle qui attend depuis le plus longtemps. En cas d'égalité, on applique une règle de départage (généralement l'indice de la tâche en ordre croissant).

### **3.2.3 Propriétés**

FIFO possède plusieurs propriétés remarquables. Elle est juste (fair) au sens où aucune tâche ne peut être indéfiniment retardée au profit d'autres. Elle est très simple à implémenter (une simple file de priorité triée par date d'arrivée) et son exécution est en  $O(n)$

---

log n) pour le tri initial. En revanche, FIFO est aveugle aux durées des opérations et aux dates d'échéance, ce qui l'expose à des situations sous-optimales : une longue opération arrivée légèrement avant une courte opération sera traitée en premier, même si cela entraîne un Makespan bien plus élevé.

*Figure 7 — Illustration du principe de la règle FIFO sur une file d'attente*

---

**Algorithm 3 :** Algorithme FIFO :

**Entrée :** Ensemble des opérations

**Debut**

```
while il reste des opérations non planifiées do
  Mettre à jour Q avec les opérations disponibles
  for chaque machine libre m E M do
    if Q non vide pour m then
      Sélectionner l'opération selon la règle choisie
      Planifier l'opération sur m
      Mettre à jour les temps de fin end if end for
    end if
  end for
  Avancer t au prochain événement
```

---

### 3.3 La Règle SPT — Shortest Processing Time

#### 3.3.1 Principe

La règle SPT (Shortest Processing Time, temps de traitement le plus court) donne la priorité à l'opération dont la durée de traitement est la plus courte parmi celles en attente sur la machine. L'idée sous-jacente est que traiter d'abord les opérations courtes libère rapidement les tâches concernées, ce qui réduit leur temps de séjour dans le système et, par extension, le Makespan global.

#### 3.3.2 Propriétés théoriques

Pour le cas d'une seule machine ( $1||\Sigma C_i$ ), il est démontré que la règle SPT est optimale pour minimiser la somme des dates de complétion  $\Sigma C_i$ . Cette propriété ne se généralise pas directement au Job-Shop multi-machine, mais SPT conserve empiriquement de bonnes performances. Elle tend à produire un faible nombre de tâches simultanément en cours (work-in-process), ce qui facilite la fluidité de la production.

---

### 3.3.3 Limites

Le principal défaut de SPT est le risque de famine (starvation) pour les longues tâches : si des opérations courtes arrivent continuellement en file d'attente, les opérations longues peuvent être indéfiniment repoussées. Dans un système Job-Shop où les durées sont très hétérogènes, cet effet peut conduire à des retards importants pour certaines tâches, même si le Makespan global est amélioré.

*Figure 8 — Illustration du principe de la règle SPT sur un exemple*

---

#### **Algorithm 2 :** Algorithme SPT :

**Entrée :** Ensemble des opérations

**Début**

Initialiser la liste des opérations disponibles Q

**while** il reste des opérations non planifiées **do**

    Mettre à jour Q avec les opérations disponibles

**for** chaque machine libre m **do**

**if** Q contient des opérations pour m **then**

            Trier les opérations de Q selon leur durée croissante

            Sélectionner l'opération ayant la plus petite durée

            Planifier cette opération sur la machine m

            Mettre à jour les temps de début et de fin

**end if**

**end for**

**end while**

**end**

---

## 3.4 La Règle LPT — Longest Processing Time

### 3.4.1 Principe

La règle LPT (Longest Processing Time, temps de traitement le plus long) est l'inverse de SPT : elle donne la priorité à l'opération dont la durée de traitement est la plus longue. La motivation originelle de LPT vient des problèmes de machines parallèles : en traitant d'abord les opérations longues, on tente d'équilibrer la charge entre les machines et d'éviter que des tâches volumineuses ne soient bloquées en fin de séquence, ce qui prolongerait inutilement le Makespan.

### 3.4.2 Propriétés et limites pour le Job-Shop

LPT produit une borne approchée garantie pour le problème de machines parallèles identiques : il est démontré que  $C_{\max}(\text{LPT}) \leq (4/3 - 1/(3m)) \times C_{\max}^*$  pour  $m$  machines parallèles. Toutefois, cette garantie ne s'étend pas au Job-Shop. En pratique, LPT tend à générer des goulots d'étranglement en début de séquence : plusieurs machines longues démarrent simultanément et se bloquent mutuellement. Les résultats expérimentaux montrent que LPT produit généralement un Makespan supérieur à celui de SPT pour le JSSP.

*Figure 9 — Illustration du principe de la règle LPT sur un exemple*

### 3.5 La Règle EDD — Earliest Due Date

#### 3.5.1 Principe

La règle EDD (Earliest Due Date, date d'échéance la plus proche) donne la priorité à la tâche dont la date d'échéance  $d_i$  est la plus proche. Elle est orientée vers la satisfaction des délais clients plutôt que vers la minimisation du Makespan. La date d'échéance représente la date à laquelle le client souhaite recevoir la tâche ; dépasser cette date constitue un retard (tardiness).

#### 3.5.2 Optimalité sur machine unique

La règle EDD est optimale pour le problème à machine unique et objectif minimisation du retard maximum :  $1||L_{\max}$ . Ce résultat, établi par Jackson (1955), est l'un des résultats fondateurs de la théorie de l'ordonnancement. Il signifie que, sur une seule machine, aucune autre règle de priorité ne peut produire un retard maximum inférieur à celui de EDD. Cette propriété ne se généralise pas directement au Job-Shop multi-machine.

#### 3.5.3 Comportement en Job-Shop

Dans un contexte Job-Shop, EDD oriente les décisions d'ordonnancement vers le respect des engagements pris envers les clients. Elle peut conduire à un Makespan légèrement supérieur à celui de SPT si les dates d'échéance et les durées opératoires ne sont pas corrélées. Cependant, si l'objectif de l'atelier est de minimiser le nombre de tâches livrées en retard ou le retard maximum, EDD constitue la règle la plus adaptée parmi les quatre étudiées.

*Figure 10 — Illustration du principe de la règle EDD sur un exemple*

### 3.6 Comparaison Théorique

| Heuristique | Critère de priorité     | Optimalité prouvée | Complexité    | Risque principal          |
|-------------|-------------------------|--------------------|---------------|---------------------------|
| FIFO        | Date d'arrivée (la plus | Aucune garantie    | $O(n \log n)$ | Makespan élevé si longues |



---

| Heuristique | Critère de priorité            | Optimalité prouvée                     | Complexité    | Risque principal                                      |
|-------------|--------------------------------|----------------------------------------|---------------|-------------------------------------------------------|
|             | ancienneté)                    |                                        |               | tâches arrivent avant les courtes                     |
| SPT         | Durée la plus courte           | Optimal pour $1 \parallel \sum C_i$    | $O(n \log n)$ | Famine des longues tâches                             |
| LPT         | Durée la plus longue           | Approché pour $P_m \parallel C_{\max}$ | $O(n \log n)$ | Goulots d'étranglement initiaux                       |
| EDD         | Date d'échéance la plus proche | Optimal pour $1 \parallel L_{\max}$    | $O(n \log n)$ | Makespan dégradé si durées et due-dates non corrélées |

Tableau 5 — Comparaison théorique des quatre heuristiques étudiées

---

## Chapitre 4 : Implémentation

### 4.1 Choix du Langage et de l'Environnement

L'implémentation du projet a été réalisée en Python en utilisant les instances de référence issues de la bibliothèque OR-Library. Un moteur d'ordonnancement basé sur une simulation événementielle (Event-Driven Scheduler) a été développé afin de respecter les contraintes du problème Job Shop Scheduling, notamment les contraintes de précédence entre les opérations et la disponibilité des machines.

Le système implémente plusieurs heuristiques basées sur des règles de priorité : FIFO (First In First Out), SPT (Shortest Processing Time), LPT (Longest Processing Time), MWKR (Most Work Remaining), LWKR (Least Work Remaining), MOR (Most Operations Remaining) et LOR (Least Operations Remaining).

Pour chaque instance, l'algorithme construit automatiquement un ordonnancement, calcule le makespan correspondant, puis vérifie la validité de la solution obtenue à travers un module de contrôle assurant :

- l'absence de chevauchement des opérations sur une même machine ;
- le respect des contraintes de précédence entre les opérations d'un même job ;
- la cohérence temporelle de l'ordonnancement généré.

Afin d'évaluer les performances des différentes heuristiques, un programme de benchmark a été développé. Celui-ci exécute automatiquement chaque heuristique sur un ensemble d'instances de référence, mesure le temps d'exécution, calcule le makespan obtenu et compare les résultats aux meilleures solutions connues (Best Known Solutions – BKS) afin de déterminer le Gap (%).

### 4.2 Architecture du Logiciel

Le logiciel développé adopte une architecture modulaire afin de séparer les différentes responsabilités du système et de faciliter la maintenance, les tests et l'extension des heuristiques. L'application est organisée en plusieurs modules spécialisés :

- `model.py` : définit les structures de données fondamentales utilisées pour représenter le problème Job Shop Scheduling, notamment les classes Job et Operation.

- 
- `dataset_loader.py` : assure le chargement et l'interprétation des instances du problème à partir des fichiers de référence de la bibliothèque OR-Library.
  - `heuristics.py` : contient l'implémentation des différentes heuristiques de priorité utilisées pour construire les ordonnancements (FIFO, SPT, LPT, MWKR, LWKR, MOR et LOR).
  - `main.py` : constitue le point d'entrée du programme. Il permet de sélectionner l'instance, l'heuristique à appliquer et les options de visualisation.
  - `verifier.py` : vérifie automatiquement la faisabilité des solutions générées en contrôlant l'absence de chevauchement des opérations sur les machines ainsi que le respect des contraintes de précédence.
  - `benchmark.py` : exécute automatiquement les campagnes expérimentales sur plusieurs instances, mesure les temps d'exécution, calcule les makespans obtenus et évalue le Gap (%) par rapport aux valeurs optimales connues.
  - `gantt.py` : génère les diagrammes de Gantt permettant de visualiser graphiquement les ordonnancements obtenus.
  - `optimal_makespans.py` : contient les valeurs optimales ou meilleures solutions connues utilisées comme référence lors de l'évaluation expérimentale.
  - `tests/` : contient les tests unitaires permettant de vérifier le bon fonctionnement des modules principaux, notamment le chargement des instances, l'exécution des heuristiques et la validation des ordonnancements.

*Figure 11 — Architecture modulaire du logiciel développé*

### 4.3 Description des Structures de Données

Les instances du problème Job Shop Scheduling sont représentées en mémoire à l'aide de structures de données simples adaptées au traitement des heuristiques d'ordonnement.

| Structure  | Type Python     | Description                                                              |
|------------|-----------------|--------------------------------------------------------------------------|
| jobs       | List[Job]       | Liste des tâches de l'instance                                           |
| operations | List[Operation] | Liste des opérations associées à une tâche                               |
| schedule   | List[dict]      | Ordonnement généré contenant les informations (job, machine, start, end) |
| OPTIMAL    | Dict[str, int]  | Valeurs optimales ou meilleures solutions                                |

---

| Structure           | Type Python | Description                                        |
|---------------------|-------------|----------------------------------------------------|
|                     |             | connues pour les instances de référence            |
| results_by_instance | Dict        | Stockage intermédiaire des résultats expérimentaux |

#### 4.4 Génération des Diagrammes de Gantt

La visualisation des ordonnancements est réalisée à l'aide de la bibliothèque Matplotlib à travers le module *gantt.py*. Les diagrammes de Gantt permettent de représenter graphiquement la séquence d'exécution des opérations sur les différentes machines.

Dans le diagramme généré :

- l'axe horizontal représente le temps d'exécution ;
- l'axe vertical représente les machines de l'atelier ;
- chaque opération est représentée sous forme d'une barre horizontale dont la longueur est proportionnelle à sa durée ;
- chaque tâche (Job) est associée à une couleur distincte afin de faciliter son identification ;
- une légende est automatiquement générée pour distinguer les différents jobs.

Les diagrammes peuvent être affichés de manière interactive ou exportés vers différents formats graphiques (PNG, PDF, SVG) afin d'être intégrés dans l'analyse expérimentale et les rapports.

---

## Chapitre 5 : Résultats Expérimentaux

### 5.1 Protocole Expérimental

Les expérimentations ont été réalisées à l'aide des instances de référence de la bibliothèque OR-Library. Les instances utilisées comprennent les jeux de données Fisher & Thompson (ft06, ft10, ft20) ainsi que plusieurs instances de Lawrence (la01 à la10).

Chaque heuristique est exécutée automatiquement par le programme benchmark.py. Afin d'obtenir des mesures fiables, le temps d'exécution est évalué sur plusieurs répétitions, puis la moyenne est calculée.

### 5.2 Critères d'évaluation

Les performances des heuristiques sont évaluées selon plusieurs indicateurs :

- Makespan (Cmax)
- Gap (%) par rapport à la meilleure solution connue
- Temps moyen d'exécution
- Faisabilité de l'ordonnancement

### 5.3 Résultats par Instance

| Instance | Heuristique | Makespan | Optimal | Gap (%) | Temps moyen (ms) |
|----------|-------------|----------|---------|---------|------------------|
| abz5     | FIFO        | 1467     | 1234    | 18.88   | 0.17             |
| abz5     | SPT         | 1352     | 1234    | 9.56    | 0.214            |
| abz5     | LPT         | 1586     | 1234    | 28.53   | 0.200            |
| abz5     | MWKR        | 1369     | 1234    | 10.94   | 0.309            |
| abz5     | LWKR        | 1735     | 1234    | 40.60   | 0.277            |
| abz5     | MOR         | 1336     | 1234    | 8.27    | 0.254            |
| abz5     | LOR         | 1559     | 1234    | 26.34   | 0.310            |

Tableau 8 — Résultats expérimentaux sur l'instance **abz5**

---

| Instance | Heuristique | Makespan | Optimal | Gap (%) | Temps moyen (ms) |
|----------|-------------|----------|---------|---------|------------------|
| abz6     | FIFO        | 1045     | 943     | 10.82   | 0.302            |
| abz6     | SPT         | 1097     | 943     | 16.33   | 0.349            |
| abz6     | LPT         | 1207     | 943     | 28.00   | 0.239            |
| abz6     | MWKR        | 987      | 943     | 4.67    | 0.250            |
| abz6     | LWKR        | 1110     | 943     | 17.71   | 0.304            |
| abz6     | MOR         | 1031     | 943     | 9.33    | 0.239            |
| abz6     | LOR         | 1144     | 943     | 21.31   | 0.278            |

Tableau 9 — Résultats expérimentaux sur l'instance **abz6**

| Instance | Heuristique | Makespan | Optimal | Gap (%) | Temps moyen (ms) |
|----------|-------------|----------|---------|---------|------------------|
| ft10     | FIFO        | 1184     | 930     | 27.31   | 0.1784           |
| ft10     | SPT         | 1074     | 930     | 15.48   | 0.2839           |
| ft10     | LPT         | 1295     | 930     | 39.25   | 0.2034           |
| ft10     | MWKR        | 1108     | 930     | 19.14   | 0.3637           |
| ft10     | LWKR        | 1334     | 930     | 43.44   | 0.3268           |
| ft10     | MOR         | 1163     | 930     | 25.05   | 0.1984           |
| ft10     | LOR         | 1352     | 930     | 45.38   | 0.1714           |

Tableau 10 — Résultats expérimentaux sur l'instance **Ft10**

Figure 16 — Histogramme comparatif des Makespan sur les trois instances

*Les résultats complets obtenus sur l'ensemble des instances sont présentés en annexe*

---

## 5.4 Analyse des Résultats

### 5.4.1 Qualité des solutions

Les résultats montrent que les heuristiques basées sur le travail restant (MWKR) ou sur le nombre d'opérations restantes (MOR) produisent généralement les meilleurs ordonnancements. Par exemple, sur l'instance **abz6**, MWKR obtient un makespan de **987**, soit un Gap de seulement **4,67 %**, ce qui constitue la meilleure performance parmi les heuristiques évaluées.

À l'inverse, LWKR et LPT présentent des écarts plus importants par rapport aux meilleures solutions connues.

### 5.4.2 Temps d'exécution

Toutes les heuristiques présentent un temps d'exécution inférieur à une milliseconde sur les instances étudiées.

Les différences de temps entre les heuristiques restent très faibles (quelques dixièmes de milliseconde), ce qui montre que le coût de calcul est négligeable comparé à la qualité des solutions produites.

### 5.4.3 Influence du choix de l'heuristique

Les résultats confirment que le choix de la règle de priorité influence directement la qualité de l'ordonnancement obtenu.

Les heuristiques **MWKR** et **MOR** fournissent généralement les meilleurs makespans, tandis que **LWKR** et **LPT** obtiennent des performances plus faibles sur la majorité des instances testées

*Tableau 11 — Synthèse comparative des quatre heuristiques (tous critères)*

---

## Conclusion Générale

Ce projet intégrateur nous a conduits à explorer de manière approfondie le problème d'ordonnancement Job-Shop, l'un des problèmes les plus représentatifs et les plus étudiés de l'optimisation combinatoire. En partant des fondements théoriques de l'ordonnancement, nous avons progressivement construit une compréhension solide du modèle Job-Shop : ses contraintes, sa complexité algorithmique, sa représentation mathématique et graphique, et ses implications industrielles.

L'étude des quatre heuristiques de répartition — FIFO, SPT, LPT et EDD — nous a permis de mettre en lumière les compromis inhérents à toute méthode approchée. Chaque règle de priorité traduit une certaine vision de ce qui est « important » dans un problème d'ordonnancement : l'équité (FIFO), la fluidité de la production (SPT), l'équilibrage des charges (LPT), ou le respect des engagements clients (EDD). Le choix d'une heuristique n'est donc pas anodin et doit être guidé par les objectifs prioritaires de l'atelier.

Sur le plan expérimental, les résultats obtenus sont convergents et cohérents avec la littérature : SPT produit systématiquement les meilleurs Makespan parmi les quatre règles testées, avec un écart moyen d'environ 9 % par rapport aux solutions de référence. EDD se révèle le meilleur compromis lorsque les délais de livraison sont une préoccupation centrale. LPT et FIFO, bien que plus performants dans d'autres contextes, s'avèrent sous-optimaux pour le Job-Shop multi-machine orienté minimisation du Makespan.

Sur le plan de l'implémentation, l'architecture modulaire retenue a facilité le développement et les tests. La séparation entre le moteur d'ordonnancement (Dispatcher), les règles de priorité (classes héritées de Heuristic) et l'analyse des résultats (module analysis) permet d'étendre facilement le logiciel à de nouvelles heuristiques sans modifier le code existant, conformément au principe ouvert/fermé (Open/Closed Principle) du génie logiciel.

Ce travail constitue une base prometteuse pour plusieurs développements futurs. En premier lieu, l'implémentation de métaheuristiques — algorithmes génétiques, recuit simulé, recherche tabou, optimisation par colonies de fourmis — permettrait d'améliorer significativement la qualité des solutions, notamment pour les grandes instances. En deuxième lieu, l'extension au Job-Shop flexible (plusieurs machines peuvent exécuter une même opération) ou au Job-Shop dynamique (arrivée de nouvelles tâches pendant l'exécution) enrichirait la modélisation pour la rapprocher davantage de la réalité industrielle. En troisième lieu, l'intégration d'une interface graphique interactive permettrait aux



---

utilisateurs de visualiser en temps réel l'évolution de l'ordonnancement et de comparer les heuristiques de façon intuitive.

Plus généralement, ce projet nous a appris à aborder un problème d'optimisation complexe avec rigueur : modéliser avec précision, choisir des méthodes adaptées aux contraintes pratiques, implémenter de façon structurée, et analyser les résultats avec esprit critique. Ces compétences, au carrefour de la recherche opérationnelle, de l'algorithmique et du génie logiciel, sont au cœur du métier d'ingénieur en systèmes de production ou en informatique décisionnelle.

---

## Références Bibliographiques

- [1] Graham, R. L., Lawler, E. L., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1979). Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey. *Annals of Discrete Mathematics*, 5, 287–326.
- [2] Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, New York.
- [3] Fisher, H., & Thompson, G. L. (1963). Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules. In J. F. Muth & G. L. Thompson (Eds.), *Industrial Scheduling* (pp. 225–251). Prentice-Hall, Englewood Cliffs, NJ.
- [4] Johnson, S. M. (1954). Optimal Two- and Three-Stage Production Schedules with Setup Times Included. *Naval Research Logistics Quarterly*, 1(1), 61–68.
- [5] Jackson, J. R. (1955). Scheduling a Production Line to Minimize Maximum Tardiness. Research Report 43, Management Science Research Project, University of California, Los Angeles.
- [6] Pinedo, M. L. (2016). *Scheduling: Theory, Algorithms, and Systems* (5th ed.). Springer International Publishing, Cham.
- [7] Blazewicz, J., Domschke, W., & Pesch, E. (1996). The Job Shop Scheduling Problem: Conventional and New Solution Techniques. *European Journal of Operational Research*, 93(1), 1–33.
- [8] Taillard, E. (1993). Benchmarks for Basic Scheduling Problems. *European Journal of Operational Research*, 64(2), 278–285.

- 
- [9] Nawaz, M., Enscore, E. E., & Ham, I. (1983). A Heuristic Algorithm for the m-Machine, n-Job Flow-Shop Sequencing Problem. *Omega*, 11(1), 91–95.
- [10] Roy, B., & Sussman, B. (1964). Les problèmes d'ordonnancement avec contraintes disjonctives. Note DS no 9, SEMA, Montrouge, France.
- [11] Garey, M.R., Johnson, D.S., & Sethi, R. (1976)